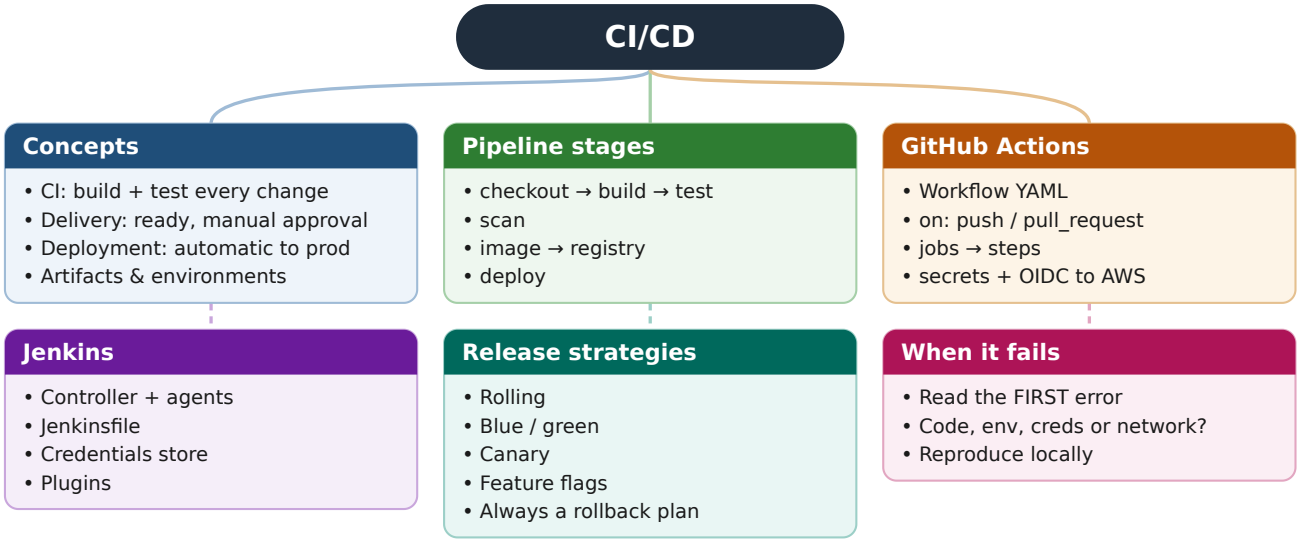
11. CI/CD

In this section: 11.1 Concepts · 11.2 GitHub Actions · 11.3 Jenkins · 11.4 Pipeline failure checklist · 11.5 Deployment strategies · 11.6 More GitHub Actions features



11.1 Concepts

Term	Meaning
Continuous Integration (CI)	Every change is automatically built and tested when pushed / on every PR
Continuous Delivery	Every passing change is ready to release; production deploy needs a manual approval
Continuous Deployment	Every passing change goes to production automatically
Pipeline	The automated sequence of stages
Stage / job / step	A phase (build, test) / a unit of work on one runner / a single command
Runner / agent	The machine that executes the jobs
Artifact	An output kept from the pipeline (binary, test report, Docker image)
Trigger	What starts the pipeline: push, PR, schedule, manual
Environment	A deployment target (dev, staging, prod), often with approval rules

```
Code push → Build → Test → Scan → Docker image → Push to registry
          → Deploy dev → (approval) → Deploy prod
```

11.2 GitHub Actions

Term	Meaning
Workflow	A YAML file in `.github/workflows/`
Event (`on:`)	Trigger: `push` , `pull_request` , `workflow_dispatch` (manual), `schedule`
Job	Runs on a runner (`runs-on: ubuntu-latest`). Jobs run in parallel unless `needs:`
Step	`uses:` a ready-made action, or `run:` a shell command
Secrets / variables	Repo or environment settings, used as `${{ secrets.NAME }}`
OIDC	The workflow gets short-lived AWS credentials by assuming an IAM role. **No stored AWS keys**

```
# .github/workflows/ci.yml
name: ci
on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read
  id-token: write          # needed for OIDC to AWS

jobs:
  build-test-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install and test
        run: |
          pip install -r requirements.txt
          pytest

      - name: Configure AWS credentials (OIDC)
        if: github.ref == 'refs/heads/main'
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-actions-ecr
          aws-region: ap-south-1

      - name: Log in to ECR
        if: github.ref == 'refs/heads/main'
        id: ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build, scan, push image
        if: github.ref == 'refs/heads/main'
        env:
          IMAGE: ${{ steps.ecr.outputs.registry }}/myapp:${{ github.sha }}
        run: |
          docker build -t "$IMAGE" .
          trivy image --exit-code 1 --severity CRITICAL "$IMAGE"
          docker push "$IMAGE"
```

Good practice: least `permissions:`, pin action versions, use OIDC instead of keys, protected `main` with required checks, environments with approvals for prod.

11.3 Jenkins

Term	Meaning
Controller	The Jenkins server: UI, scheduling, configuration
Agent	A machine that runs builds (keep builds off the controller)
Job / Pipeline	A configured task / a pipeline defined in code
Jenkinsfile	Pipeline as code, stored in the repo
Plugins	Add features (Git, Docker, credentials, Kubernetes)
Credentials store	Encrypted secrets used in pipelines via `withCredentials` / `credentials()`

```
// Jenkinsfile (declarative)
pipeline {
  agent any
  environment {
    IMAGE = "myuser/myapp:${env.BUILD_NUMBER}"
  }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Test')     { steps { sh 'pip install -r requirements.txt && pytest' } }
    stage('Build')    { steps { sh 'docker build -t $IMAGE .' } }
    stage('Push') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'dockerhub',
                         usernameVariable: 'USER', passwordVariable: 'PASS')]) {
          sh 'echo "$PASS" | docker login -u "$USER" --password-stdin && docker push $IMAGE'
        }
      }
    }
    stage('Deploy') {
      when { branch 'main' }
      steps { sh './deploy.sh $IMAGE' }
    }
  }
  post {
    failure { echo 'Pipeline failed: check the stage logs' }
    always  { cleanWs() }
  }
}
```

Run Jenkins locally: `docker run -d -p 8080:8080 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts`. The first admin password is in `/var/jenkins_home/secrets/initialAdminPassword`.

	GitHub Actions	Jenkins
Hosting	Managed by GitHub (or self-hosted runners)	You install and maintain it
Config	YAML in `.github/workflows`	Groovy `Jenkinsfile`
Best for	Repos on GitHub, quick setup	Complex/on-prem setups, many legacy enterprises

11.4 Pipeline failure checklist

1. Which stage failed? Read the **first** error in the log, not the last.

2. Is it the code (tests failing), the environment (missing tool/version), credentials/permissions, or the network?
3. Does it pass locally with the same command and versions?
4. Did anything change recently (dependency version, secret expired, runner image)?
5. Fix the cause and push again. Never disable tests to get green.

11.5 Deployment strategies

Strategy	How it works	Pros	Cons
Recreate	Stop old version, start new	Simple	Downtime
Rolling update	Replace instances a few at a time	No downtime, default in Kubernetes	Old and new run together for a while
Blue/green	Run new version (green) next to old (blue), switch traffic at once	Instant switch and rollback	Double resources during release
Canary	Send a small % of traffic to the new version, watch metrics, then increase	Limits the blast radius	Needs good monitoring and traffic splitting
Feature flags	New code is deployed but switched off; turn on per user/percentage	Separates deploy from release	Flag management overhead

Rollback plan: every deployment needs one (previous image tag, `kubectl rollout undo`, `helm rollback`, `git revert` in GitOps). Database changes must be backward-compatible so you can roll back the app.

11.6 More GitHub Actions features

Feature	Example / purpose
Caching	`actions/cache` or `cache: pip` in setup actions → faster builds
Matrix builds	`strategy: matrix: python: ["3.11", "3.12"]` → test several versions in parallel
Job dependencies	`needs: test` → deploy only after tests pass
Environments	`environment: production` → required reviewers and environment-specific secrets
Reusable workflows	`uses: org/repo/.github/workflows/build.yml@v1` → one standard pipeline for many repos
Artifacts	`actions/upload-artifact` / `download-artifact` → pass build outputs between jobs
Concurrency	`concurrency: deploy-prod` → only one deploy at a time
Path filters	`on: push: paths: ["app/**"]` → run only when relevant files change

REMEMBER

- **Code → Build → Test → Scan → Image → Registry → Deploy.**
- Prefer **OIDC** (short-lived credentials) over stored cloud keys.
- Never disable tests to make a pipeline green.